INTRODUCTION TO Machine Learning

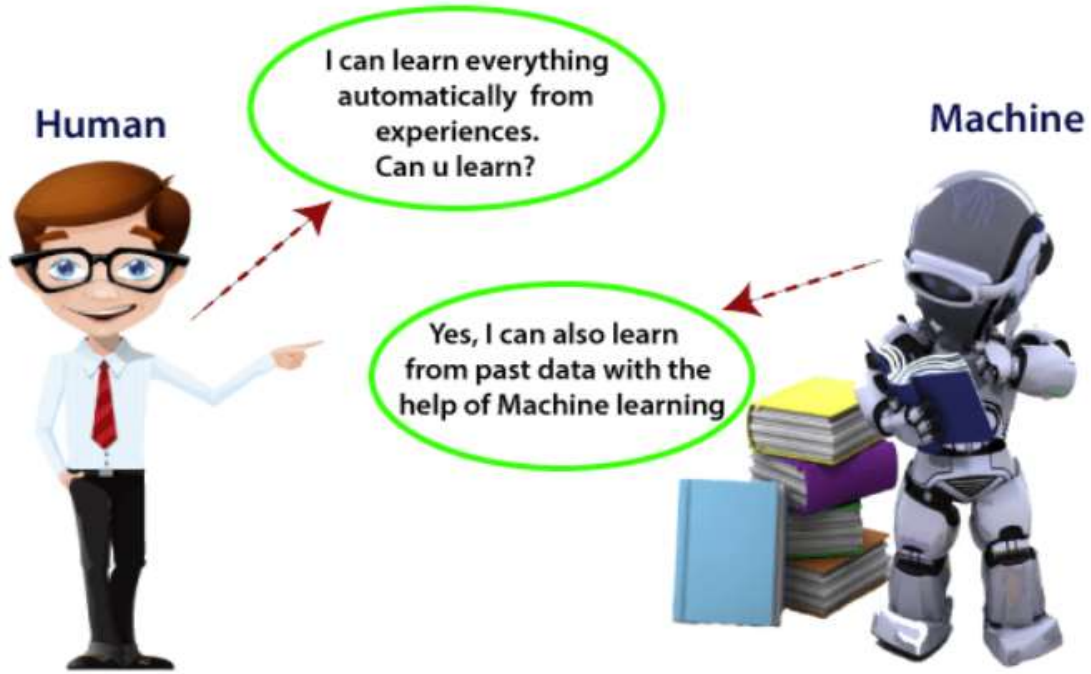


Introduction to Machine Learning

Machine Learning (ML) is a branch of artificial intelligence (AI) that enables systems to learn and improve from experience without being explicitly programmed. By analyzing data, ML algorithms identify patterns and make decisions or predictions, making it a cornerstone of modern technology

Key Concepts

1. **Data**
   - The foundation of ML is data. Algorithms learn by analyzing datasets, which can be structured (e.g., spreadsheets) or unstructured (e.g., images, audio).
2. **Features**
   - Features are measurable properties or characteristics of the data, used as inputs for the ML model.
3. **Labels**
   - Labels are the desired outputs in supervised learning, helping the model learn a relationship between features and outcomes.



Steps in a Machine Learning Workflow

1. **Problem Definition**
   - Understand and define the problem to be solved.
2. **Data Collection**
   - Gather relevant and high-quality data.
3. **Data Preprocessing**
   - Clean and prepare data by handling missing values, scaling, and encoding features.
4. **Model Selection**
   - Choose an algorithm based on the problem type and data.
5. **Training**
   - Train the model on the dataset.
6. **Evaluation**
   - Assess the model's performance using metrics like accuracy, precision, recall, or F1 score.
7. **Tuning**
   - Optimize the model by adjusting hyperparameters.
8. **Deployment**
   - Deploy the trained model to make real-world predictions.



Why Python is Used in Machine Learning

1. **Ease of Use**
    - Python's simple and readable syntax makes it easy for developers and researchers to focus on solving ML problems rather than struggling with the programming language.
2. **Extensive Libraries and Frameworks**
    - Python offers powerful ML libraries such as **TensorFlow**, **PyTorch**, **scikit-learn**, and **Keras**, simplifying complex ML tasks.
3. **Rich Ecosystem for Data Handling**
    - Libraries like **NumPy**, **Pandas**, and **Matplotlib** enable efficient data manipulation, analysis, and visualization, essential for ML workflows.
4. **Platform Independence**
    - Python code runs on various platforms, making it suitable for developing and deploying ML applications seamlessly.

5. **Integration Capabilities**
    - Python easily integrates with other languages, databases, and frameworks, allowing it to fit into diverse tech stacks.
6. **Active Community Support**
    - A large and active Python community provides abundant resources, tutorials, and solutions for ML developers.
7. **Versatility**
    - Python supports end-to-end ML workflows, from data preprocessing to model deployment and performance monitoring.
8. **Adoption in Research and Industry**
    - Its widespread use in academic research and industry ensures a wealth of pre-built tools, datasets, and pretrained models.

TYPES OF MACHINE LEARNING

- Machine Learning algorithm are divided into four types, based on the following criteria.
- Whether or not they are trained with human supervision
    1. Supervised Learning
    2. Unsupervised Learning
    3. Semi supervised Learning
    4. Reinforcement Learning



SUPERVISED LEARNING

- In supervised learning, the training set that provides data for an algorithm contains a convenient solution called labels.
- A typical supervised learning task is classification.
- Spam filter is a good example. It has been trained on with sample emails along with a label defined (spam or ham). After learning from existing emails it must classify new emails.
- Another common task is prediction for example car price prediction. For prediction it offers a number of features like mileage, age, make, etc. called predictors. With the help of predictors new car prize will be predicted. That kind of work called regression.



SUPERVISED LEARNING ALGORITHMS

- k-Nearest Neighbors
- Linear Regression
- Logistic Regression
- Support Vector Machines (SVMs)
- Decision Trees and Random Forests
- Neural networks



UNSUPERVISED LEARNING

In Unsupervised Learning training data is unlabeled. In this type of machine learning, system learn without a teacher. Some important Unsupervised Learning algorithm are given below:

1. Clustering(K-Means)
2. Anomaly detection and novelty detection (One-class SVM, Isolation Forest)
3. Visualization and dimensionality reduction (PCA)
4. Association rule learning

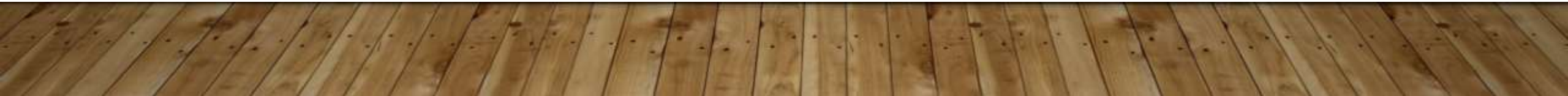
SEMI- SUPERVISED LEARNING

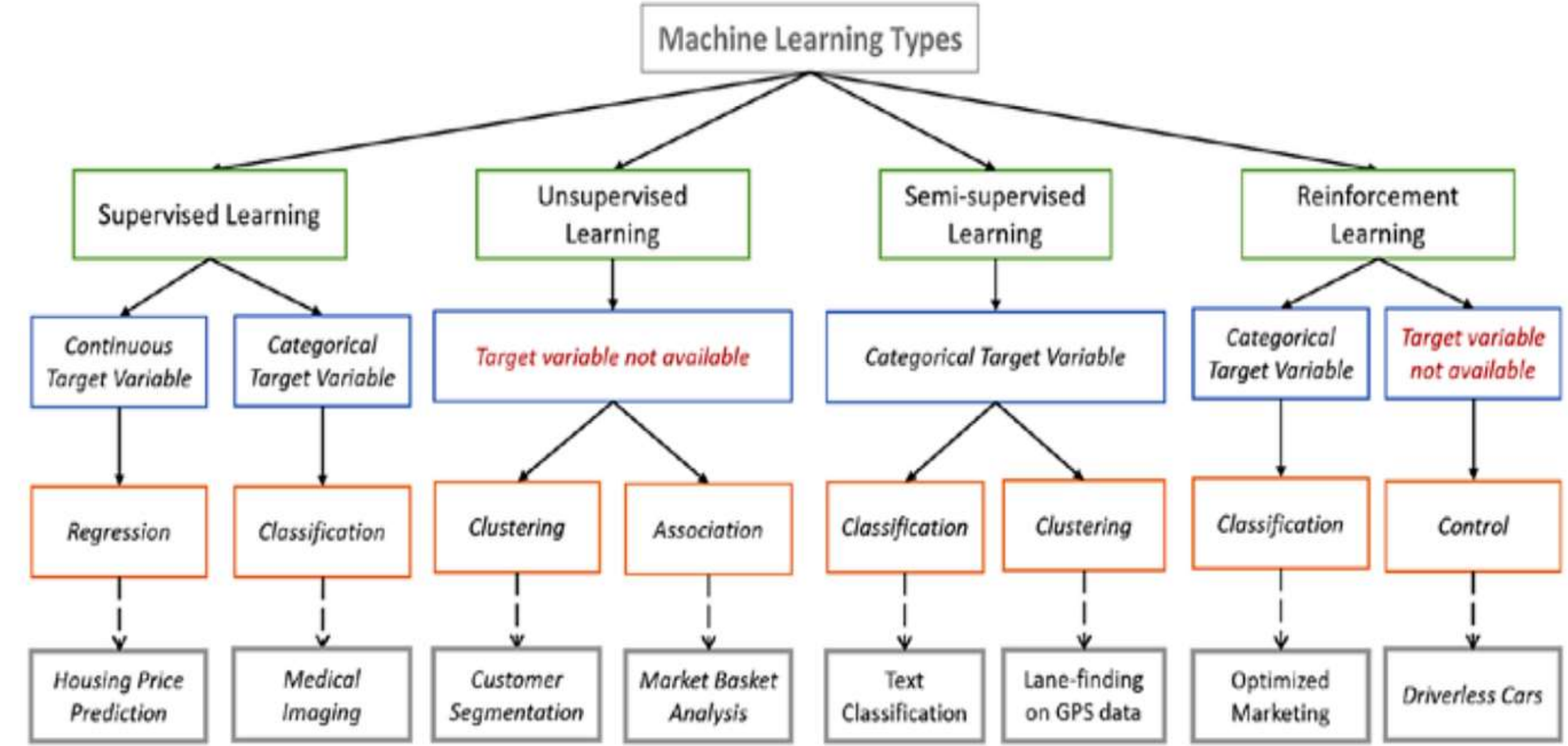
- Labelling data is very costly and time-consuming process.
- Some algorithms can deal with data that's partially labeled means few labeled instances and few unlabeled instances. This is called semi-supervised learning.
- Some photo-hosting services, such as Google Photos, are good examples of this. Semi-supervised learning algorithms are combinations of unsupervised and supervised algorithms e.g. deep belief networks (DBNs).



REINFORCEMENT LEARNING

- Reinforcement learning is a whole other animal.
- This learning system is known as Agent.
- In reinforcement learning an action is taken by observing the environment and in return get reward.
- It learns by itself which is best policy to get the most reward over time.
- For example, many robots implement Reinforcement Learning algorithms to learn how to walk. DeepMind's AlphaGo program is also a good example of Reinforcement Learning




Machine Learning Types
Supervised Learning
Unsupervised Learning
Semi-supervised Learning
Reinforcement Learning
Continuous Target Variable
Categorical Target Variable
Target variable not available
Categorical Target Variable
Categorical Target Variable
Target variable not available
Regression
Classification
Clustering
Association
Classification
Clustering
Classification
Control
Housing Price Prediction
Medical Imaging
Customer Segmentation
Market Basket Analysis
Text Classification
Lane-finding on GPS data
Optimized Marketing
Driverless Cars

Applications of Machine Learning

1. **Healthcare**
   - **Disease Diagnosis**: Detecting diseases like cancer using ML-based imaging.
   - **Personalized Medicine**: Recommending treatments based on patient history.
   - **Drug Discovery**: Accelerating research through predictive algorithms.
2. **Finance**
   - **Fraud Detection**: Identifying unusual patterns in transactions.
   - **Credit Scoring**: Assessing loan eligibility and creditworthiness.
   - **Algorithmic Trading**: Using ML to make investment decisions.
3. **Retail and E-commerce**
   - **Recommendation Systems**: Suggesting products based on user behavior (e.g., Amazon, Netflix).
   - **Dynamic Pricing**: Adjusting prices based on demand and competitor analysis.
   - **Inventory Management**: Predicting stock requirements using sales data.
4. **Transportation**
   - **Autonomous Vehicles**: Self-driving cars using ML for navigation and safety (e.g., Tesla).
   - **Traffic Prediction**: Real-time congestion forecasting (e.g., Google Maps).
   - **Ride-Sharing**: Optimizing routes and pricing (e.g., Uber).

5. **Entertainment**
    - **Content Recommendation**: Tailoring playlists and shows (e.g., Spotify, YouTube).
    - **Game AI**: Enhancing player experience with smarter game opponents.
6. **Agriculture**
    - **Crop Monitoring**: Detecting diseases and optimizing yields through ML-powered drones.
    - **Weather Prediction**: Improving forecasts for farming decisions.
7. **Cybersecurity**
    - **Anomaly Detection**: Identifying and mitigating cyber threats.
    - **Spam Filtering**: Blocking phishing emails and malicious content.
8. **Customer Support**
    - **Chatbots**: Automating responses to common queries (e.g., AI-powered assistants).
    - **Sentiment Analysis**: Understanding customer emotions from reviews or feedback.
9. **Education**
    - **Personalized Learning**: Adapting content to individual student needs.
    - **Plagiarism Detection**: Ensuring academic integrity.
10. **Energy**

- **Smart Grids**: Optimizing energy distribution and predicting outages.
- **Renewable Energy Forecasting**: Improving solar and wind energy utilization.



Uses of Machine Learning

1. **Automation**
   - **Use**: Automating repetitive and manual tasks.
   - **Example**: Robotic Process Automation (RPA) in customer support, where ML-powered bots handle inquiries.
2. **Prediction**
   - **Use**: Predicting future outcomes based on historical data.
   - **Example**: Weather forecasting or stock market predictions using ML models.
3. **Personalization**
   - **Use**: Customizing user experiences.
   - **Example**: Netflix recommending shows and movies based on a user's watch history.
4. **Pattern Recognition**
   - **Use**: Identifying patterns in data to make informed decisions.
   - **Example**: Detecting credit card fraud by recognizing unusual spending patterns.

5. **Optimization**
    - **Use:** Improving systems for better efficiency.
    - **Example:** Dynamic pricing on e-commerce platforms adjusts prices based on demand and competition.
6. **Image and Speech Recognition**
    - **Use:** Interpreting visual and audio data.
    - **Example:** Facial recognition in smartphones or voice assistants like Alexa and Siri.
7. **Anomaly Detection**
    - **Use:** Spotting outliers in data.
    - **Example:** Detecting network intrusions or equipment malfunctions in manufacturing.
8. **Classification**
    - **Use:** Categorizing data into predefined groups.
    - **Example:** Email spam filters categorizing emails as "spam" or "not spam."

NumPy

NumPy is an open-source Python library that provides support for large, multi-dimensional arrays and matrices

It also have a collection of high-level mathematical functions to operate on arrays

Features of NumPy

- Provides powerful, multidimensional array objects (ndarrays).
- Supports advanced slicing and indexing techniques.
- Reduces the need for explicit loops with efficient array operations.
- Easily integrates with other libraries like pandas, matplotlib, and SciPy.
- Useful linear algebra, Fourier transform, and random number capabilities



In [5]:
```
# Installation and Importing Numpy
!pip install numpy
import numpy as np
```

```
Requirement already satisfied: numpy in c:\users\kapil\anaconda\lib\site-packages (1.23.5)
```

In [6]:
```
# Creating NumPy Arrays From Lists
arr = np.array([1, 2, 3, 4])
print(arr)
```

```
[1 2 3 4]
```

In [40]:
```
# Creating 2D Arrays
arrd = np.array([[1, 2], [3, 4]])
print(arrd)
```

```
[[1 2]
 [3 4]]
```

In [17]:
```
## NumPy Array Attributes
# Shape: Dimensions of the array
arrd.shape
```

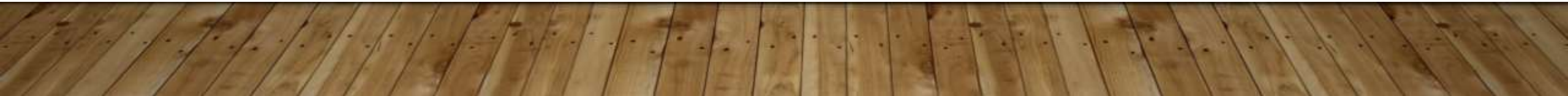
Out[17]:
```
(2, 2)
```

In [15]:
```
# Size: Total number of elements
arr.size
```

Out[15]:
```
4
```

In [12]:
```
# Dtype: Data type of elements
arr.dtype
```

Out[12]:
```
dtype('int32')
```



In [19]:
```
## Array Operations
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
result = arr1 + arr2  # Element-wise addition
print(result)
```

```
[5 7 9]
```

In [25]:
```
## NumPy Functions
# Arange (Creating ranges)
arr = np.arange(0, 10, 3)   # starting from 0 and end at 10 having gap 3
print(arr)
```

```
[0 3 6 9]
```

In [23]:
```
# Linspace (Creating evenly spaced numbers):
arr = np.linspace(0, 10, 5)
print(arr)
```

```
[ 0.   2.5  5.   7.5 10. ]
```

In [26]:
```
# Indexing and Slicing
#  Indexing
arr = np.array([1, 2, 3, 4, 5])
print(arr[0])  # Access first element
```

```
1
```

In [27]:
```
# Slicing
arr = np.array([1, 2, 3, 4, 5])
print(arr[1:4])  # Get elements from index 1 to 3
```

```
[2 3 4]
```



In [41]:
```
# Reshaping Arrays (R*C)
arr = np.array([1, 2, 3, 4, 5, 6])

reshaped_arr = arr.reshape(2, 3)
print(reshaped_arr)


#rr=arr.reshape(6,1)
#print(rr)
```

```
[[1 2 3]
 [4 5 6]]
```

In [39]:
```
# Common NumPy Functions
arr = np.array([1, 2, 3, 4, 5])
print("Sum of all element is ",np.sum(arr))
print("Mean of all element is ",np.mean(arr))
print("Standared Deviation of all element is ",np.std(arr))
print("Variation of all element is ",np.var(arr))
print("Minimum value from all elememt is ",np.min(arr))
print("Maximum value from all elememt is ",np.max(arr))
```

```
Sum of all element is  15
Mean of all element is  3.0
Standared Deviation of all element is  1.4142135623730951
Variation of all element is  2.0
Minimum value from all elememt is  1
Maximum value from all elememt is  5
```

In [ ]:

Pandas

- Pandas is a powerful and open-source Python library.
- The Pandas library is used for data manipulation and analysis.
- Pandas consist of data structures and functions to perform efficient operations on data.
- Pandas is well-suited for working with **tabular data**, such as **spreadsheets** or **SQL tables**.

Features of Pandas

•Data set cleaning, merging, and joining.

•Easy handling of missing data (represented as NaN) in floating point as well as non-floating point data.

•Columns can be inserted and deleted from DataFrame and higher-dimensional objects.

•Powerful group by functionality for performing split-apply-combine operations on data sets.

•Data Visualization



In [2]:
```
# Installing and Importing pandas
!pip install pandas
import pandas as pd
```

```
Requirement already satisfied: pandas in c:\users\kapil\anaconda\lib\site-packages (1.5.3)
Requirement already satisfied: pytz>=2020.1 in c:\users\kapil\anaconda\lib\site-packages (from pandas) (2022.7)
Requirement already satisfied: numpy>=1.21.0 in c:\users\kapil\anaconda\lib\site-packages (from pandas) (1.23.5)
Requirement already satisfied: python-dateutil>=2.8.1 in c:\users\kapil\anaconda\lib\site-packages (from pandas) (2.8.2)
Requirement already satisfied: six>=1.5 in c:\users\kapil\anaconda\lib\site-packages (from python-dateutil>=2.8.1->pandas) (1.1
6.0)
```

In [5]:
```
# Pandas Data Structures
# Series: One-dimensional labeled array
s = pd.Series([1, 2, 3])
s
```

Out[5]:
```
0    1
1    2
2    3
dtype: int64
```

In [4]:
```
# DataFrame: Two-dimensional labeled data
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df
```

Out[4]:

	A	B
0	1	3
1	2	4

In [14]:
```
# Loading Data
# From CSV
df = pd.read_csv(r'C:\Users\Kapil\OneDrive\Desktop\Practical\heart (4).csv')
df
```

Out[14]:

	age	gender	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	target
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3	0
1	53	1	0	140	203	1	0	155	1	3.1	0	0	3	0
2	70	1	0	145	174	0	1	125	1	2.6	0	0	3	0
3	61	1	0	148	203	0	1	161	0	0.0	2	1	3	0
4	62	0	0	138	294	1	1	106	0	1.9	1	3	2	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1020	59	1	1	140	221	0	1	164	1	0.0	2	0	2	1
1021	60	1	0	125	258	0	0	141	1	2.8	1	1	3	0
1022	47	1	0	110	275	0	0	118	1	1.0	1	1	2	0
1023	50	0	0	110	254	0	0	159	0	0.0	2	0	2	1
1024	54	1	0	120	188	0	1	113	0	1.4	1	1	3	0

1025 rows × 14 columns

In [18]:
```
# From Excel
df = pd.read_excel(r'C:\Users\Kapil\OneDrive\Desktop\Practical\simple_data.xlsx')
df
```

Out[18]:

	Name	Age	City
0	Kapil	25	Umarkhed
1	Aniket	30	Yavatmal
2	Sanket	35	Pune

In [16]:
```
# From Json
df = pd.read_json(r'C:\Users\Kapil\OneDrive\Desktop\Practical\simple_data.json')
df
```

Out[16]:

	Name	Mark
0	Kapil	78
1	Aniket	87

In [20]:
```
# Basic Operations
# Viewing data:
df.head()
df.tail()
```

Out[20]:

	Name	Age	City
0	Kapil	25	Umarkhed
1	Aniket	30	Yavatmal
2	Sanket	35	Pune
3	Ram	40	Matheran

In [21]:
```
# Summary statistics:
df.describe()
```

Out[21]:

	Age
count	4.000000
mean	32.500000
std	6.454972
min	25.000000
25%	28.750000

In [25]:
```
# Shape and
df.shape
```

Out[25]:
```
(4, 3)
```

In [23]:
```
# data types:
df.dtypes
```

Out[23]:
```
Name     object
Age       int64
City     object
dtype: object
```

In [28]:
```
# Data Cleaning

# Handling missing values:
df.dropna()
df.fillna(0)
```

Out[28]:

	Name	Age	City
0	Kapil	25	Umarkhed
1	Aniket	30	Yavatmal
2	Sanket	35	Pune
3	Ram	40	Matheran

In [29]:
```
# Renaming columns
df.rename(columns={'City': 'Address'})
```

Out[29]:

	Name	Age	Address
0	Kapil	25	Umarkhed
1	Aniket	30	Yavatmal

In [32]:
```
# Data Selection
df['Name']
```

Out[32]:
```
0     Kapil
1    Aniket
2    Sanket
3       Ram
Name: Name, dtype: object
```

In [34]:
```
df.loc[0]
```

Out[34]:
```
Name       Kapil
Age           25
City    Umarkhed
Name: 0, dtype: object
```

In [33]:
```
df.iloc[0, 0]
```

Out[33]:
```
'Kapil'
```

In [37]:
```
# Data Transformation
df['Country'] = 'India'
df['Age_Double'] = df['Age'] * 2
df.to_csv('updated_data.csv', index=False)
df
```

Out[37]:

	Name	Age	City	Country	Age_Double
0	Kapil	25	Umarkhed	India	50
1	Aniket	30	Yavatmal	India	60
2	Sanket	35	Pune	India	70
3	Ram	40	Matheran	India	80

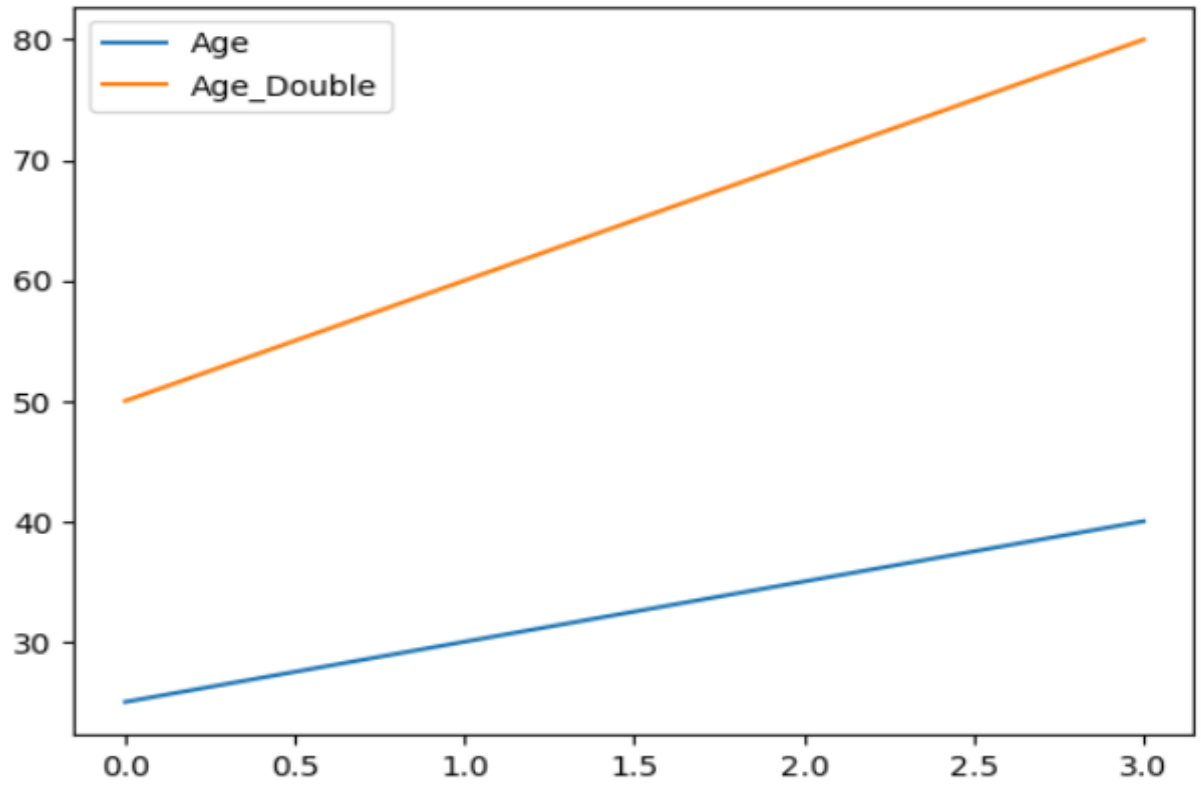
In [38]:
```
# Filtering rows:
df[df['Age'] > 32]
```

Out[38]:

	Name	Age	City	Country	Age_Double
2	Sanket	35	Pune	India	70
3	Ram	40	Matheran	India	80

In [39]:
```
# Data Visualization
df.plot()
```

Out[39]: <Axes: >



Matplotlib: Overview and Explanation

Matplotlib is a popular Python library for creating static, interactive, and animated visualizations. It is widely used for data visualization in scientific computing and machine learning.



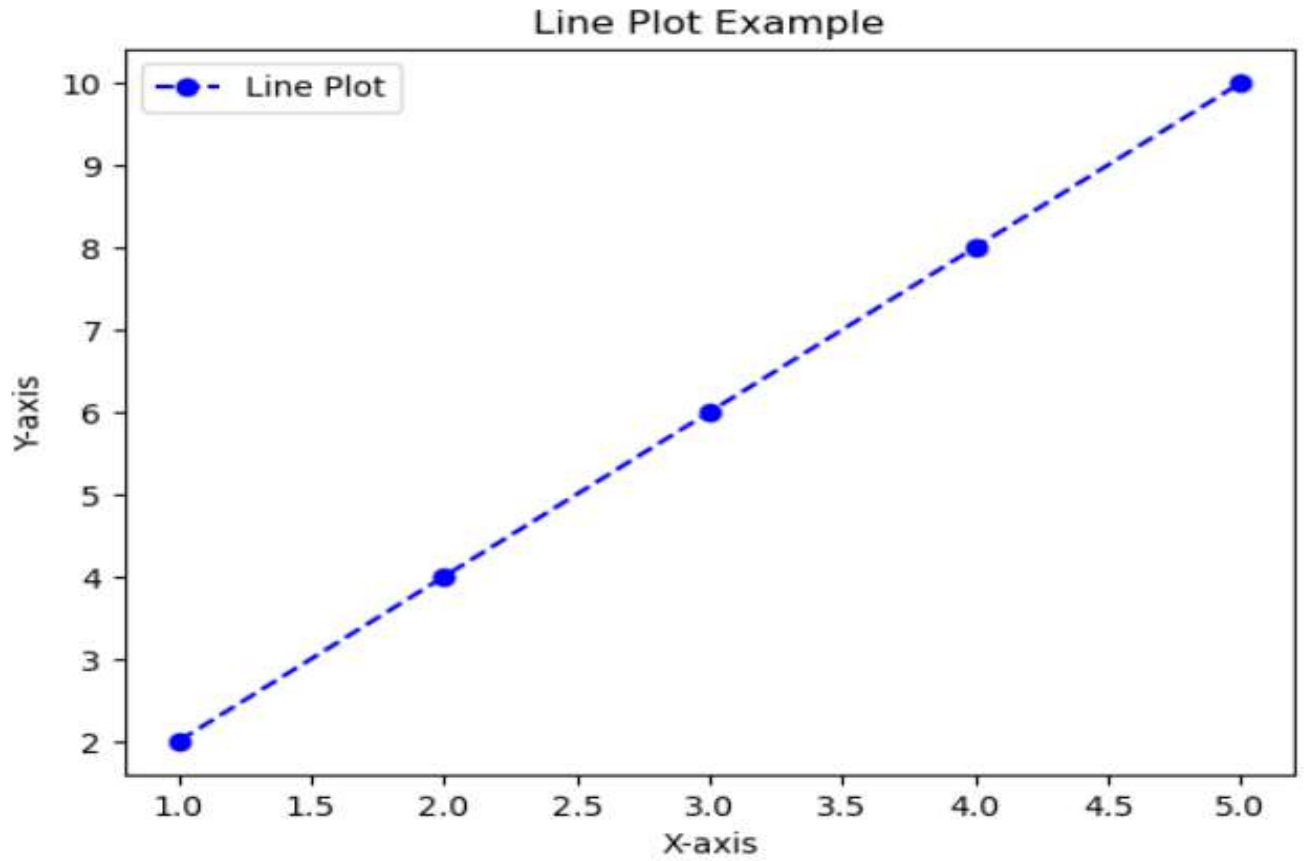
Key Features of Matplotlib:

1. **Wide Range of Plots:** Supports line plots, bar charts, scatter plots, histograms, pie charts, and more.
2. **Highly Customizable:** You can control the appearance of plots (axes, labels, colors, styles, etc.).
3. **Integration:** Works well with libraries like NumPy and pandas.
4. **Multiple Outputs:** Can save plots as images in various formats (PNG, PDF, etc.).
5. **Interactive Features:** Allows zooming, panning, and adding interactive elements.

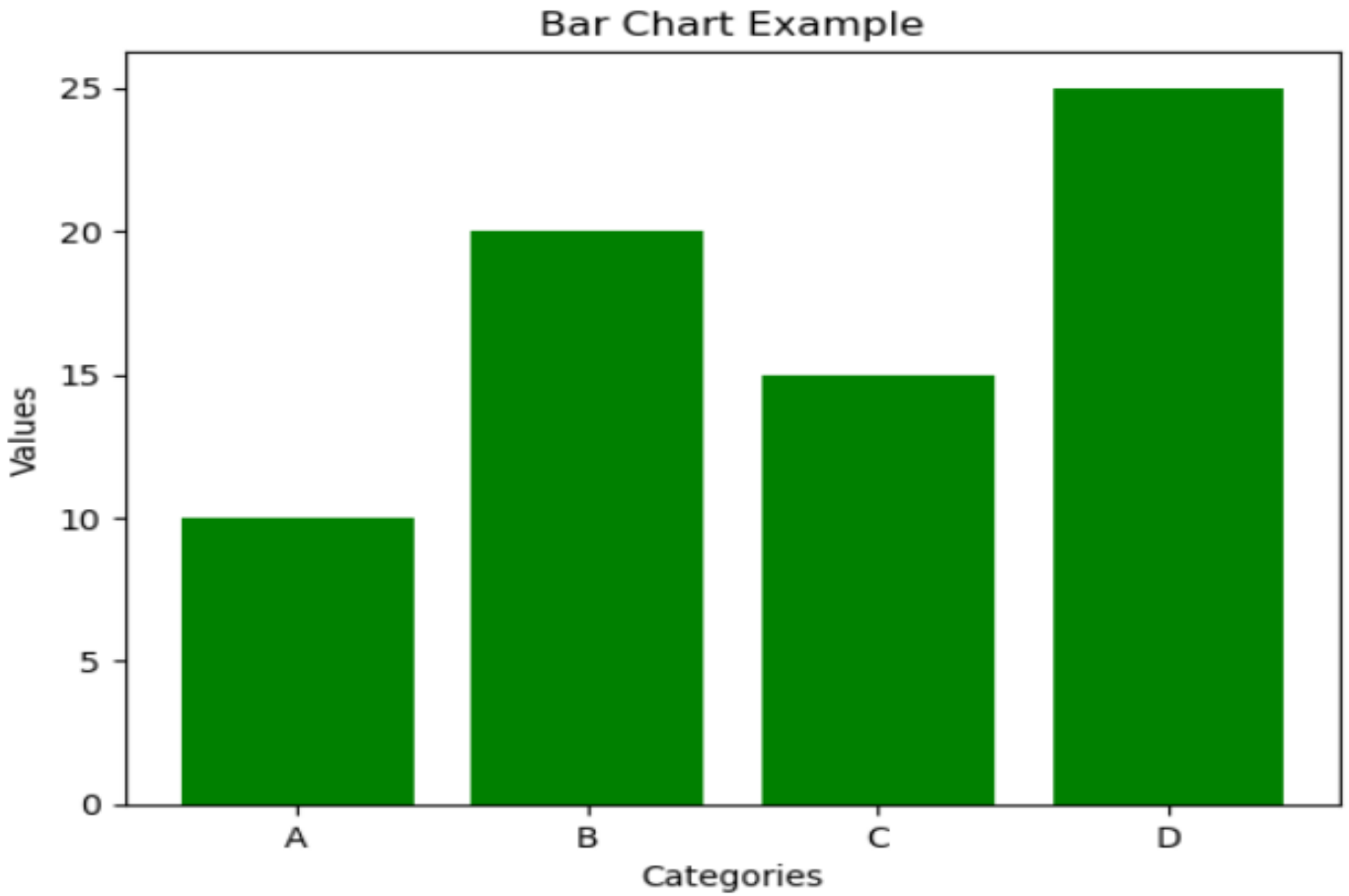
Basic Steps to Use Matplotlib:

1. Import the library.
2. Prepare the data.
3. Create a plot using functions from `matplotlib.pyplot`.
4. Customize the plot (titles, labels, legends, etc.).
5. Display or save the plot.

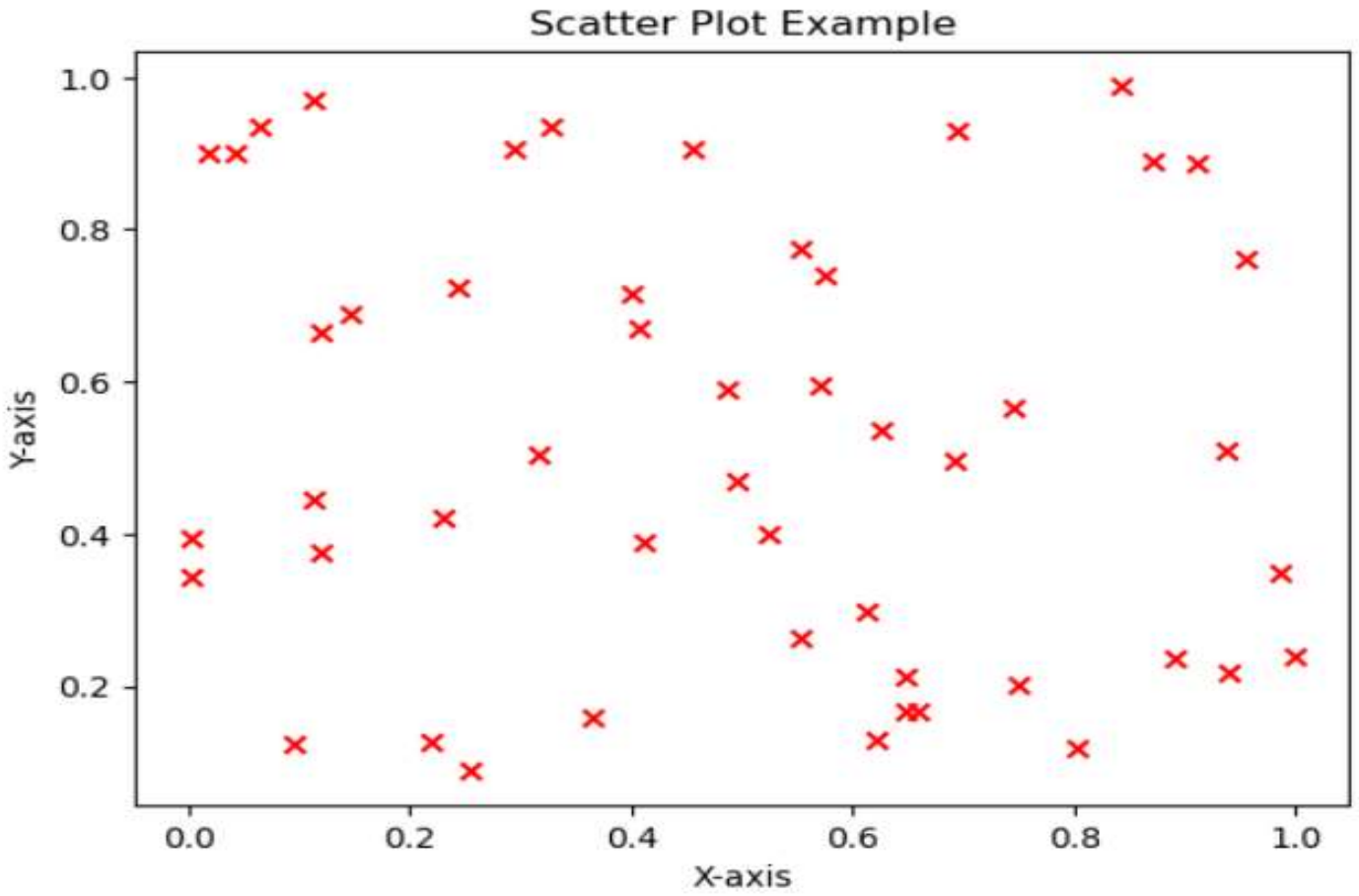
In [1]:
```
# Line Plot
import matplotlib.pyplot as plt
# Data
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
plt.plot(x, y, label='Line Plot', color='blue', linestyle='--', marker='o')
plt.title('Line Plot Example')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.legend()
plt.show()
```



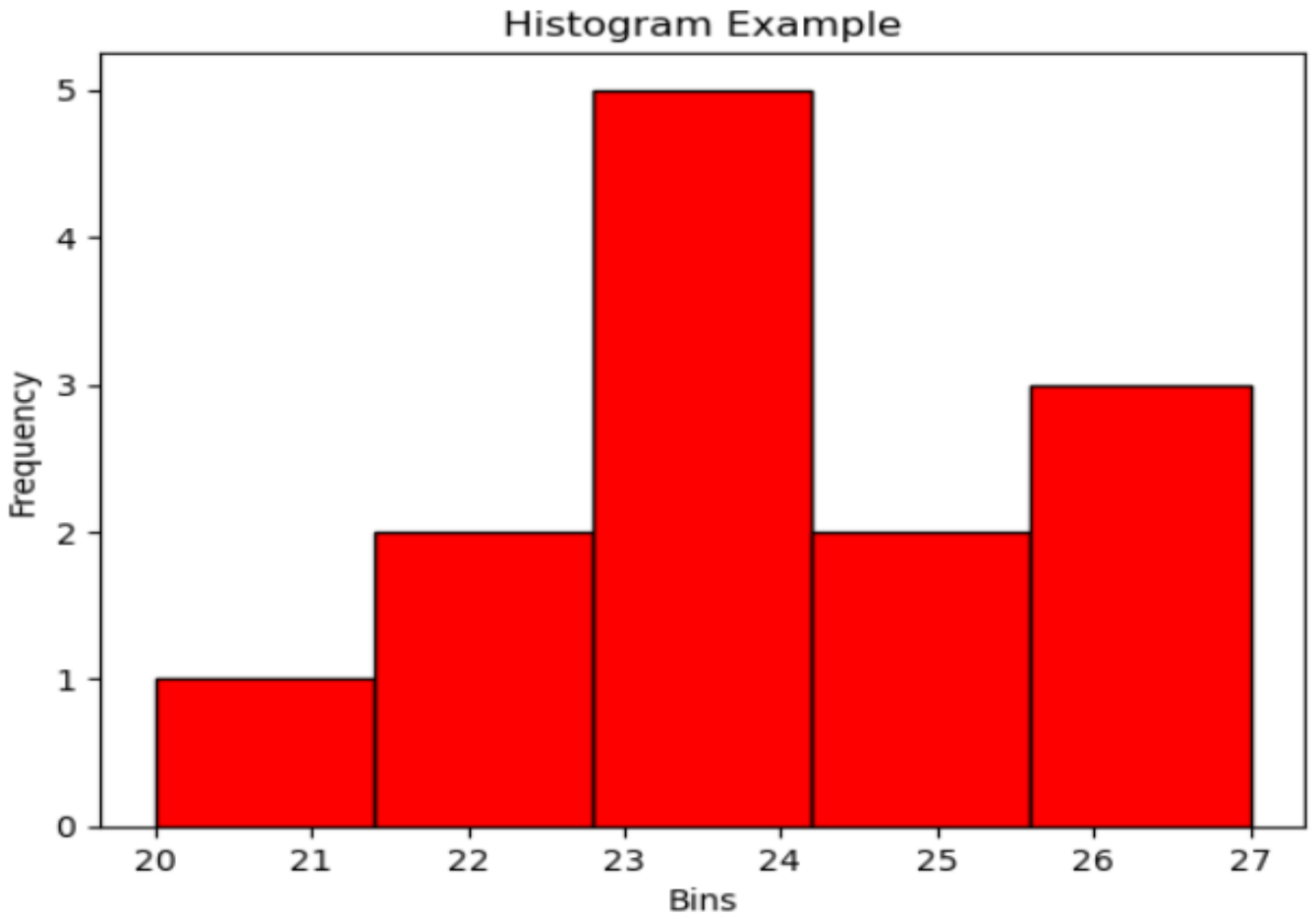
[2]:
```
# Bar Chart
# Data
categories = ['A', 'B', 'C', 'D']
values = [10, 20, 15, 25]
plt.bar(categories, values, color='green')
plt.title('Bar Chart Example')
plt.xlabel('Categories')
plt.ylabel('Values')
plt.show()
```



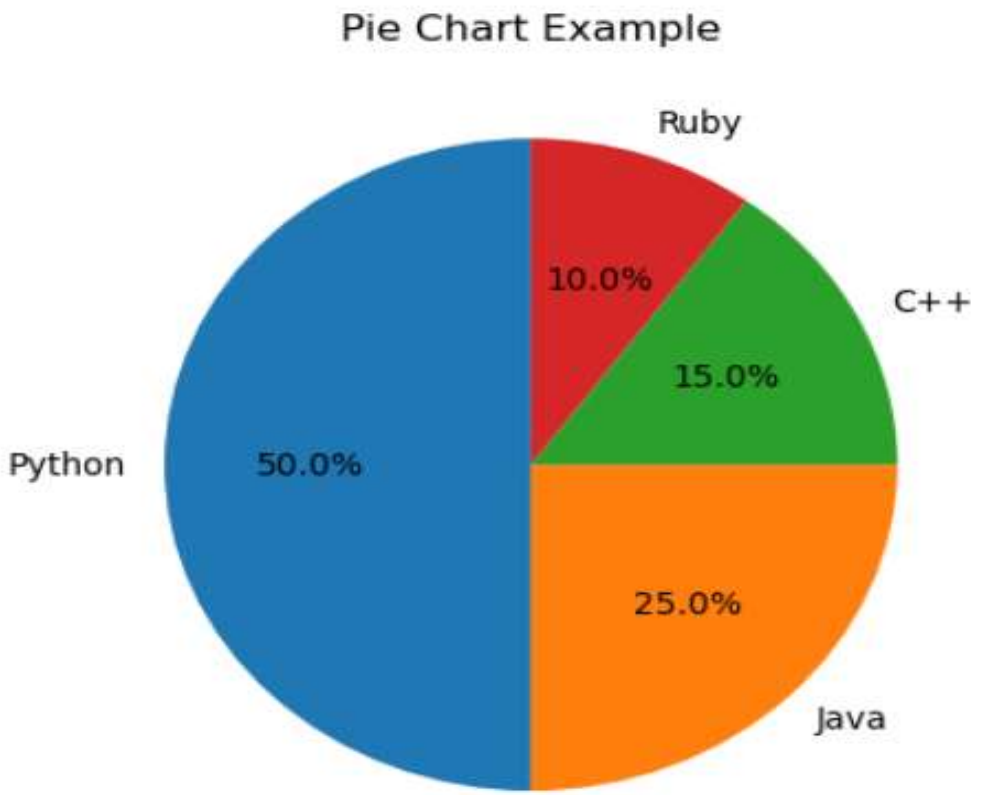
[3]:
```
# Scatter Plot
import numpy as np
x = np.random.rand(50)
y = np.random.rand(50)
plt.scatter(x, y, color='red', marker='x')
plt.title('Scatter Plot Example')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.show()
```



[7]:
```
# Histogram
# Data
data = [20, 23, 22, 24, 26, 25, 23, 24, 26, 27, 23, 22, 25]
plt.hist(data, bins=5, color='red', edgecolor='black')
plt.title('Histogram Example')
plt.xlabel('Bins')
plt.ylabel('Frequency')
plt.show()
```



[8]:
```
# Pie Chart
# Data
labels = ['Python', 'Java', 'C++', 'Ruby']
sizes = [50, 25, 15, 10]
# Plot
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
# Customizations
plt.title('Pie Chart Example')
# Show plot
plt.show()
```



SciPy: Definition

SciPy (Scientific Python) is an open-source Python library used for scientific and technical computing. It builds on **NumPy** and provides additional functionality for optimization, integration, interpolation, eigenvalue problems, algebraic equations, and other advanced mathematical and scientific computations.

Uses of SciPy

1. **Scientific Computation:** Used in fields like physics, engineering, and biology for advanced scientific tasks.
2. **Optimization:** Solves linear, nonlinear, and complex optimization problems.
3. **Integration:** Performs numerical integration and solving differential equations.
4. **Linear Algebra:** Handles matrix operations, decompositions, and solving linear equations.
5. **Signal Processing:** Performs tasks like filtering, Fourier transforms, and convolution.
6. **Statistics:** Offers advanced statistical tests, probability distributions, and statistical functions.

Advantages of SciPy

1. **Comprehensive Functionality:**
    - Covers a wide range of scientific computations, eliminating the need for multiple libraries.
2. **Integration with NumPy:**
    - Seamlessly works with NumPy arrays, ensuring high performance for numerical tasks.
3. **Open Source and Well-Maintained:**
    - Constantly updated and widely used in academia and industry, with a large community of users and contributors.

Disadvantages of SciPy

1. Steep Learning Curve:
    - Requires a good understanding of mathematical concepts, which might be challenging for beginners.
2. Performance Overhead:
    - Not as fast as lower-level languages like C or Fortran for certain high-performance tasks.
3. Limited Visualization:
    - Focuses on computation, relying on other libraries (like Matplotlib or Seaborn) for visualizations.

Real-World Examples of SciPy

1. **Optimization in Logistics:**
    - Companies use SciPy's optimization tools (e.g., `scipy.optimize`) to minimize transportation costs or optimize delivery routes.
2. **Signal Processing in Audio Engineering:**
    - SciPy's signal processing module (`scipy.signal`) is used to filter noise and analyze sound frequencies in audio processing applications.
3. **Statistical Analysis in Clinical Trials:**
    - SciPy's statistical functions (`scipy.stats`) help analyze experimental data and validate hypotheses in medical research.

In [9]:
```python
#  Optimization Example (Minimizing a Function)
# Used for finding the minimum or maximum of functions.\


from scipy.optimize import minimize

# Define a quadratic function
def objective(x):
    return x**2 + 2*x + 1

# Minimize the function
result = minimize(objective, x0=0)  # Initial guess at x=0
print("Optimal Value:", result.x)
```

```
Optimal Value: [-1.00000001]
```

In [10]:
```python
# Integration Example
# Used for numerical integration of functions.


from scipy.integrate import quad

# Define a function to integrate
def func(x):
    return x**2

# Integrate the function from 0 to 2
result, _ = quad(func, 0, 2)
print("Integral:", result)
```

```
Integral: 2.666666666666667
```

In [11]:
```python
# Interpolation (scipy.interpolate)
# Used to find intermediate values between data points.
from scipy.interpolate import interp1d
import numpy as np

# Sample data points
x = [0, 1, 2, 3, 4]
y = [0, 2, 4, 6, 8]

# Create an interpolation function
f = interp1d(x, y)

# Interpolate for a new value
print("Interpolated value at x=2.5:", f(2.5))
```

```
Interpolated value at x=2.5: 5.0
```

In [12]:
```python
# Statistics (scipy.stats)
# Provides a range of statistical functions.
from scipy.stats import norm

# Mean and standard deviation of a normal distribution
mean, std = 0, 1

# Probability Density Function (PDF) at x=0
pdf_value = norm.pdf(0, loc=mean, scale=std)
print("PDF at x=0:", pdf_value)

# Generate random samples from the distribution
samples = norm.rvs(loc=mean, scale=std, size=5)
print("Random samples:", samples)
```

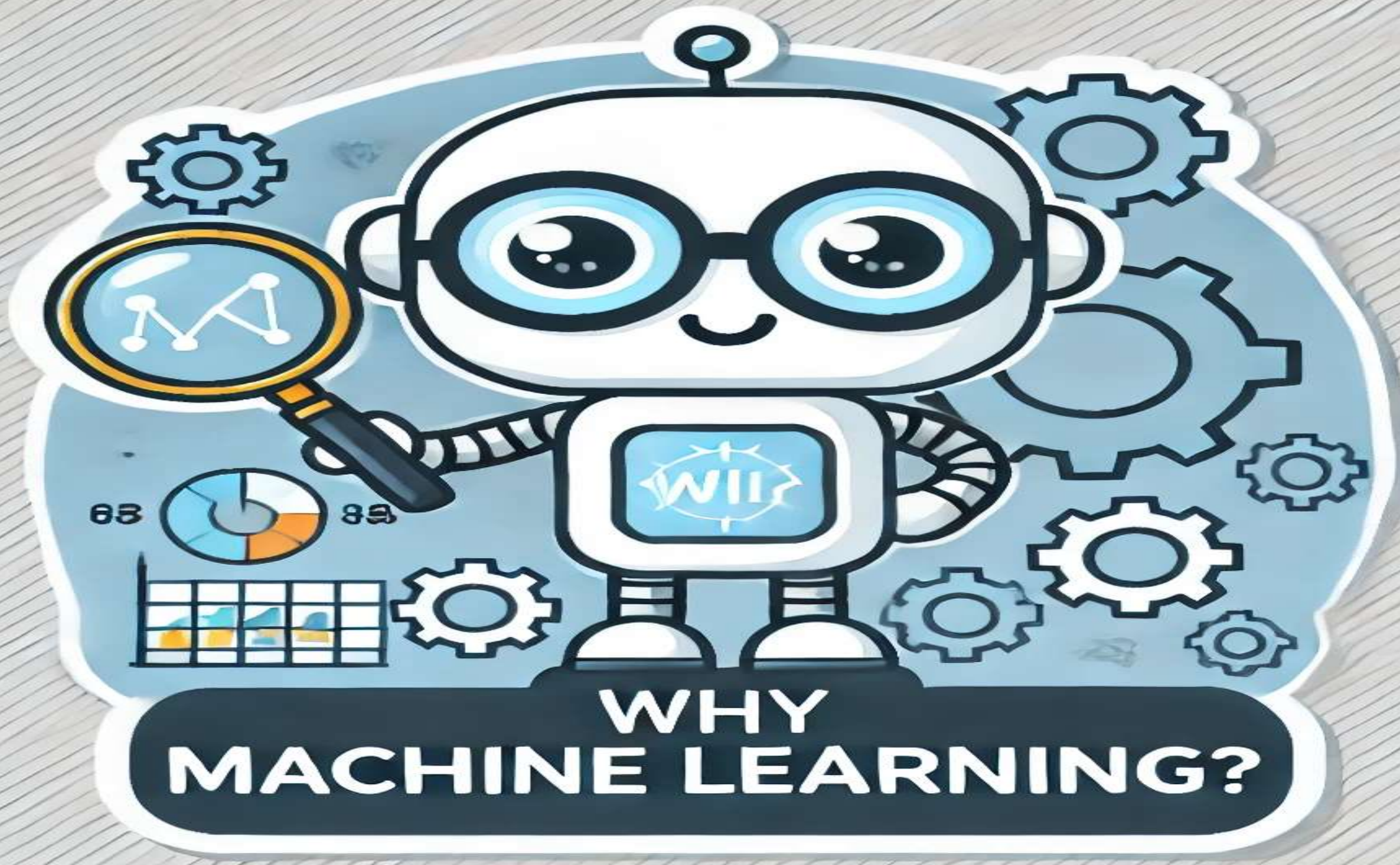
```
PDF at x=0: 0.3989422804014327
Random samples: [-0.20527412  0.27756472 -1.98792725  0.83528391 -0.59505973]
```

In [13]:
```
# Linear Algebra (scipy.linalg)
# Provides advanced matrix operations and decompositions.
from scipy.linalg import solve
# Coefficient matrix
A = [[3, 2], [1, 4]]
# Right-hand side
b = [8, 7]
# Solve Ax = b
x = solve(A, b)
print("Solution to the equations:", x)
```

```
Solution to the equations: [1.8 1.3]
```

In [14]:
```
# Signal Processing Example
# Provides tools for filtering, convolution, and signal analysis.
from scipy.signal import butter, filtfilt
import numpy as np
# Create a Butterworth low-pass filter
def butter_lowpass_filter(data, cutoff, fs, order=5):
    nyquist = 0.5 * fs
    normal_cutoff = cutoff / nyquist
    b, a = butter(order, normal_cutoff, btype='low', analog=False)
    y = filtfilt(b, a, data)
    return y
# Sample noisy data
fs = 30.0  # Sampling frequency
data = np.sin(2 * np.pi * 0.75 * np.arange(0, 1, 1/fs)) + 0.5 * np.random.randn(len(np.arange(0, 1, 1/fs)))
# Apply low-pass filter
filtered_data = butter_lowpass_filter(data, cutoff=1.5, fs=fs)
print("Filtered data:", filtered_data)
```

```
Filtered data: [-0.34490895 -0.1090929   0.12387137  0.34784937  0.55696327  0.74580282
  0.90960331  1.04438975  1.14708969  1.21561611  1.24892032  1.24701262
  1.21094772  1.14277187  1.04542971  0.92262976  0.77866982  0.61822696
  0.44612151  0.26706945  0.08544198 -0.09494581 -0.27099524 -0.44041465
 -0.6017293  -0.75420963 -0.89772849 -1.03256867 -1.15920894 -1.27811971]
```


WHY
MACHINE LEARNING?



Thank You!

For your attention